Nama : Gaitsha Azzahra Devitrya

Kelas : 2A-TNK

NIM : 231344012

Minggu : 11

Hari, Tanggal : Jumat, 9 Mei 2025

STRING INSTRUCTIONS

String instructions adalah kumpulan instruksi yang digunakan untuk memproses data dalam bentuk urutan (string) byte atau word, seperti memindahkan, membandingkan, mengisi, atau memindai blok memori.

Instruksi-instruksi dapat menangani data secara efisien menggunakan register khusus seperti SI (Source Index), DI (Destination Index), dan CX (counter) tanpa perlu menggunakan loop eksplisit.

Instruksi CLD (Clear Direction Flag) dan STD (Set Direction Flag) adalah dua instruksi penting dalam bahasa assembly mikroprosesor 8086 yang berkaitan langsung dengan pengaturan arah eksekusi operasi string. Keduanya memanipulasi flag arah (*Direction Flag* atau DF) yang merupakan bagian dari register status *FLAGS*, dan memiliki peran besar dalam menentukan bagaimana instruksi-instruksi string seperti MOVS, LODS, STOS, SCAS, dan CMPS menavigasi area memori.

- CLD (Clear Directions) : digunakan untuk *menghapus* Direction Flag, yaitu menyetelnya ke nilai 0. Ketika DF bernilai 0, maka setiap operasi string akan dilakukan dalam arah *increment* atau maju—dari alamat memori rendah ke alamat memori yang lebih tinggi.
  Contoh program :

```
01 TITLE TRANSFER 20 DATA
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 BM1 DB 30H,31H,32H,33H,34H,35H,36H,37H,38H,39H,3AH,3BH,3CH,3DH,3EH,3FH,40H,41H,42H,43H
06 BM2 DB 20 DUP(0)
07 .CODE
08 MAIN PROC
09     MOV AX,@DATA
10     MOV DS,AX
11     MOV ES,AX
12     CLD
13     LEA SI, BM1
14     LEA DI, BM2
15     mov cx,20
16     rep movsb
17
18  MOV AH,4CH
19     INT 21H
20
21 MAIN ENDP
22 END MAIN
23
```

- STD (Set Directions) : menyetel Direction Flag menjadi 1. Ini menyebabkan semua operasi string dilakukan dalam arah *decrement* atau mundur—dari alamat memori tinggi ke alamat memori yang lebih rendah.
  Contoh program :

```
TITLE TRANSFER 20 DATA
.MODEL SMALL
.STACK 100H
.DATA
BM1 DB 30H,31H,32H,33H,34H,35H,36H,37H,38H,39H,3AH,3BH,3CH,3DH,3EH,3FH,40H,41H,42H,43H
BM2 DB 20 DUP(0)
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX
    MOV ES,AX
    STD
    LEA SI, BM1+19
    LEA DI,, BM2+19
    MOV CX,20
    REP MOVSB

 MOV AH,4CH
    INT 21H

MAIN ENDP
END MAIN
```

1. Buat program untuk mencopy data dari blok memori 1 (BM1) ke blok memori 2(BM2) yang masing-masing terdiri dari 20 lokasi memori.
   - Cara 1 : menyalin dari data atas ke data bawah (Arah 0)

```
TITLE TRANSFER 20 DATA
.MODEL SMALL
.STACK 100H
.DATA
BM1 DB 30H,31H,32H,33H,34H,35H,36H,37H,38H,39H,3AH,3BH,3CH,3DH,3EH,3FH,40H,41H,42H,43H
BM2 DB 20 DUP(0)
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    MOV DI,0
    MOV DL,0
L1: MOV DL,[BM1+DI]
    MOV [BM2+DI],DL
    INC DI
    CMP DI,20
    JNZ J1
          |

 MOV AH,4CH
    INT 21H

MAIN ENDP
END MAIN
```

variables

size: byte elements: 21

edit show as: hex

```
BM1     30h, 31h, 32h, 33h, 34h, 35h, 36h, 37h, 38h, 39h, 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh, 40h, 41h, 42h, 43h, 30h
BM2     30h, 31h, 32h, 33h, 34h, 35h, 36h, 37h, 38h, 39h, 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh, 40h, 41h, 42h, 43h, 00h
```

   - Cara 2 : menyalin dari data bawah ke data atas (Arah 1)

```
TITLE TRANSFER 20 DATA
.MODEL SMALL
.STACK 100H
.DATA
BM1 DB 30H,31H,32H,33H,34H,35H,36H,37H,38H,39H,3AH,3BH,3CH,3DH,3EH,3FH,40H,41H,42H,43H
BM2 DB 20 DUP(0)
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    MOV DI,13H
    MOV DL,0
L1: MOV DL,[BM1+DI]
    MOV [BM2+DI],DL
    DEC DI
    JNS L1


 MOV AH,4CH
    INT 21H

MAIN ENDP
END MAIN
```

variables

size: byte elements: 21

edit show as: hex

```
BM1     30h, 31h, 32h, 33h, 34h, 35h, 36h, 37h, 38h, 39h, 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh, 40h, 41h, 42h, 43h, 30h
BM2     30h, 31h, 32h, 33h, 34h, 35h, 36h, 37h, 38h, 39h, 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh, 40h, 41h, 42h, 43h, 00h
```

2. Menampillkan karakter/ gambar pada layar monitor

Pada sistem mikroprosesor berbasis Intel 8086, tampilan teks pada layar monitor dikendalikan melalui memori video yang terletak pada alamat segmen B800H. Dalam mode teks standar, layar monitor memiliki dimensi 80 kolom dan 25 baris, sehingga mampu menampilkan total 2000 karakter (80 × 25) sekaligus.

Setiap karakter yang ditampilkan menempati dua byte dalam memori video: satu byte untuk kode ASCII karakter, dan satu byte lagi untuk atribut tampilan seperti warna latar belakang dan warna teks (foreground dan background color). Dengan demikian, memori video memerlukan 4000 byte (2000 × 2) untuk menampilkan seluruh layar. Alamat memori dimulai dari segmen B800H:0000H untuk baris pertama dan kolom pertama, lalu berlanjut secara linear hingga baris terakhir dan kolom terakhir.

```
01 TITLE BENDERA
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 .CODE
06 MAIN PROC
07     MOV AX,0B800H ;MENDEKLARASIKAN ALAMAT SEGMENT
08     MOV DS,AX
09     MOV CX,960 ; 12<BARIS> * 80<KOLOM> YANG AKAN DIGUNAKAN
10     MOV SI,0   ; GAMBAR AKAN DIMULAI DARI KOLOM 0
11 LUP:MOV [SI],0C020H ; MERAH
12     ADD SI,2
13     LOOP LUP
14     MOV CX,960
15 LUP1:MOV [SI],0F020H;PUTIH
16     ADD SI,2
17     LOOP LUP1
18
19     MOV AH,4CH
20     INT 21H
21 MAIN ENDP
22 END MAIN
```



Praktikum :

Program Seleksi Data dan Pengurutan

1. Buat diagram alir dan program untuk menghitung banyaknya data bernilai positif (data>0) dari 20 data berikut. Simpan hasil perhitungan pada register CL.

.DATA

BIL DB 23H, 77H, 1CH, 9, 0CCH, 44H, 6EH, 77H, 0, 01BH

DB 0FFH, 0D2H, 6CH, 81H, 33H, 22H, 0A6H, 28H, 12H, 4

```
01 TITLE SOAL 1
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 BIL DB 23H,77H,1CH,9H,0CCH,44H,6EH,77H,0,01BH
06     DB 0FFH,0D2H,6CH,81H,33H,22H,0A6H,28H,12H,4H
07 .CODE
08 MAIN PROC
09     MOV AX,@DATA
10     MOV DS,AX
11
12     MOV SI,0
13     MOV CL,0
14
15 LUP:MOV BL,[BIL+SI]
16     CMP BL,0
17     JLE SK0
18     ADD CL,1
19 SK0:ADD SI,1
20     CMP SI,14H
21     JB LUP
22
23     MOV AH,4CH
24     INT 21H
25     MAIN ENDP
26 END MAIN
```

2. Buat diagram alir dan program untuk menghitung banyaknya data bernilai negatif (data<0) dari 20 data berikut. Simpan hasil perhitungan pada register DH.

.DATA

BIL DB 23H, 77H, 1CH, 9, 0CCH, 44H, 6EH, 77H, 0, 01BH

DB 0FFH, 0D2H, 6CH, 81H, 33H, 22H, 0A6H, 28H, 12H, 4

```
01 TITLE SOAL 2
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 BIL DB 23H,77H,1CH,9H,0CCH,44H,6EH,77H,0,01BH
06     DB 0FFH,0D2H,6CH,81H,33H,22H,0A6H,28H,12H,4H
07 .CODE
08 MAIN PROC
09     MOV AX,@DATA
10     MOV DS,AX
11
12     MOV SI,0
13     MOV DH,0
14
15 LUP:MOV AL,[BIL+SI]
16     CMP AL,0
17     JGE SK0
18     ADD DH,1
19 SK0:ADD SI,1
20     CMP SI,14H
21     JB LUP
22
23     MOV AH,4CH
24     INT 21H
25     MAIN ENDP
26 END MAIN
```